SENA · TECNÓLOGO EN ANÁLISIS Y DESARROLLO DE SOFTWARE

Documento de evidencias

Frontend en React del proyecto QR Empresarial

PROYECTO	QR Empresarial — perfiles empresariales con código QR dinámico
APRENDIZ	Jean Carlos Orozco Hernández
FICHA	3336250
GUÍAS	Frontend N.º 1 y N.º 2, más los ocho módulos del proyecto
REPOSITORIO	github.com/JECAOR/qr-empresarial
VERSIÓN	v3.0.0
FECHA	31 de octubre de 2026

Todas las capturas de este documento se tomaron contra el servidor real, con los datos de `database/seed.sql`, y se regeneran con un guion versionado.

1. Conceptos fundamentales utilizados

El frontend es una aplicación de una sola página construida con **React 19** y **Vite 8**, que consume la API REST del propio proyecto. No accede nunca a la base de datos: pide y envía datos por HTTP, y es el backend quien decide qué se puede hacer.

Componentes, props y composición

Cada pieza de interfaz es una función que devuelve JSX. Lo que cambia entre usos llega por **props**: `TarjetaContenido` recibe el título, el detalle y el tipo de bloque, y decide con una condición si se dibuja como enlace o como texto. Así la misma pieza sirve para los seis bloques del perfil sin repetir código.

Estado y efectos

`useState` guarda lo que cambia mientras la pantalla está abierta y avisa a React de que debe volver a dibujar. `useEffect` sirve para lo que ocurre fuera del dibujado —pedirle los datos al servidor— y su lista de dependencias decide cuándo se repite: con `[dias]`, solo al cambiar el periodo de las estadísticas.

Cada efecto que pide datos devuelve una función de limpieza. Sin ella, si se consulta un negocio y antes de que llegue la respuesta se consulta otro, la primera podría pisar a la segunda.

> **Dos peticiones en desarrollo no son un error.** `StrictMode` ejecuta cada efecto dos veces a propósito, para que salte a la vista un efecto que no sabe limpiarse. En producción se ejecuta una sola vez.

Contexto

La sesión —quién eres, tu rol, entrar y salir— vive en un **contexto** (`AuthContext`) en lugar de pasarse por props de pantalla en pantalla. Al abrir la aplicación no se confía en lo que hay guardado en el navegador: se le pregunta al servidor con `GET /usuarios/me`, de modo que un token caducado o revocado se detecta enseguida.

Enrutado y rutas protegidas

React Router decide qué pantalla se dibuja según la dirección. `RutaProtegida` exige sesión y, cuando hace falta, un rol.

> **El guardia no es seguridad.** Quien escriba la dirección a mano o llame a la API con `curl` se lo salta entero. La seguridad la aplica Express en cada endpoint: una cuenta de empresa recibe `403` aunque consiga pintar la pantalla. Lo que hace el guardia es no ofrecer puertas que van a dar error.

Separación en capas

Ninguna pantalla llama a `fetch`. Todas pasan por un servicio, y todos los servicios por un único `apiClient` que pone la dirección base, el tipo de contenido y el token, y convierte un error del servidor en una excepción con su mensaje. Si mañana cambia la forma de autenticarse, se toca un archivo.

2. Componentes implementados y su justificación

2.1 Estructura del proyecto

```
src/
  context/AuthContext.jsx        La sesión: quién eres, entrar, salir y refrescar
  components/
    Layout/                      Barra de navegación común
    RutaProtegida/               Exige sesión y, si hace falta, un rol
    Formulario/CampoTexto        Etiqueta, campo y ayuda, en una sola pieza
    FichaNegocio/                Datos del negocio; omite los campos vacíos
    TarjetaContenido/            Un bloque: enlace o texto, según su tipo
    contenido/ListaBloques       Los bloques con su orden y sus acciones
    contenido/FormularioBloque   Alta y edición de un bloque
    usuarios/TablaUsuarios       El listado; en móvil se vuelve fichas
    usuarios/FormularioUsuario   Alta y edición de una cuenta
    perfil/SelectorTema          El color de marca
    estadisticas/GraficaActividad La gráfica diaria, en SVG
    admin/DetalleCuenta          La actividad reciente de una cuenta
  pages/                         Trece pantallas
  services/                      Once servicios, uno por módulo de la API
```

2.2 Por qué cada decisión

COMPONENTE	JUSTIFICACIÓN
apiClient	Única puerta de salida hacia la API. El ejemplo de la guía comprueba `body.success`; esta API no tiene ese campo, y fiarse de él haría que todo pareciera correcto. Manda el código HTTP.
AuthContext	La sesión la necesitan la barra, el guardia y cuatro pantallas. Pasarla por props obligaría a atravesar componentes que no la usan.
RutaProtegida	Un solo sitio donde se decide quién pasa. Recuerda a dónde iba el usuario para devolverlo ahí después de entrar.
CampoTexto	Las cuatro pantallas de cuenta repetían el mismo bloque de etiqueta, campo y texto de ayuda.
GraficaActividad	SVG dibujado a mano, sin librería: son dos series de entre siete y noventa puntos, y Chart.js con su envoltorio para React pesa más que toda la aplicación. El `<title>` de cada punto da la información al pasar el cursor, que es cosa del navegador.
DetalleCuenta	Va en un `<dialog>` para que el navegador se encargue del foco, de `Escape` y de bloquear lo de detrás.
TablaUsuarios	Bajo 780 px cada fila se convierte en una ficha con el nombre de la columna delante de su valor. Una tabla de siete columnas en un móvil obliga a desplazar la pantalla a lo ancho para alcanzar los botones.

2.3 Dependencias

Tres en producción y cinco de desarrollo. **Ninguna librería de interfaz**: los estilos, la gráfica y los diálogos son propios.

PAQUETE	VERSIÓN	PARA QUÉ
`react` / `react-dom`	19.3	La biblioteca y su puente con el navegador
`react-router-dom`	7.18	Enrutado y rutas protegidas
`vite`	8.3	Servidor de desarrollo y empaquetado
`@vitejs/plugin-react`	6.1	Compilación de JSX y recarga en caliente
`oxlint`	1.88	Análisis estático

Resultado: **95 KB comprimidos** para trece pantallas.

3. Evidencia de integración con la API

El frontend consume **38 de los 40 endpoints** de la API REST del proyecto. Los dos restantes son `GET /api/health`, que solo comprueba que el servidor responde, y `POST /api/publico/:slug/eventos`, que registra los escaneos y las visitas: lo llama la vista pública que sirve Express, no esta aplicación, porque si lo hiciera inflaría las estadísticas cada vez que el dueño abriera su propio panel.

3.1 Endpoints por módulo

MÓDULO	ENDPOINTS	PANTALLA
Autenticación	`/auth/login`, `/logout`, `/registro`, `/recuperar-contrasena`, `/restablecer/:token`, `/restablecer-contrasena`	Entrar, Registro, Recuperar, Restablecer
Cuenta propia	`/usuarios/me` (GET, PUT, DELETE), `/me/contrasena`, `/me/preferencias` (GET, PUT)	Configuración
Usuarios	`/usuarios` (GET, POST), `/:id` (GET, PUT, DELETE), `/:id/estado`	Gestión de usuarios
Perfil	`/perfiles/me` (GET, PUT), `/me/tema`	Mi negocio
Contenido	`/contenido` (GET, POST), `/categorias`, `/orden`, `/:id` (PUT, DELETE), `/:id/estado`	Contenido
Código QR	`/qr` (GET, POST), `/estado`, `/imagen`	Código QR
Vista pública	`/publico/:slug`	Perfil público
Estadísticas	`/estadisticas`	Actividad
Administración	`/admin/resumen`, `/admin/usuarios`, `/:id`, `/reportes/usuarios`	Administración

3.2 Lo que hubo que adaptar del ejemplo de las guías

Las guías usan SICE como proyecto orientador y advierten que cada equipo debe comprobar el contrato real de su API. Al hacerlo aparecieron ocho diferencias, todas comprobadas contra el servidor con `curl` antes de escribir una sola pantalla:

EL EJEMPLO ASUME	ESTA API RESPONDE
Campo `success` en el cuerpo	No existe: manda el código HTTP
`data.user` y `expiresIn`	`data.usuario`, sin `expiresIn`
`role` como objeto `{id, name}`	`rol` como cadena
`{email, password}` al entrar	`{correo, contrasena}`
Cuatro roles	Dos: `empresa` y `administrador`
`/users/:id/status`	`/usuarios/:id/estado`
Estado como `1` o `0`	Como palabra: `activo` o `restringido`
El CRUD abierto a varios roles	Solo `administrador`

> Copiar el ejemplo tal cual habría dado `contenido.map is not a function` y una pantalla sin rol con la que no se puede proteger nada. La guía lista ese error entre los frecuentes.

3.3 Comprobación de las peticiones

La colección de Postman del proyecto trae dos carpetas dedicadas a este frontend, con **46 peticiones y 44 pruebas automáticas** que recorren los mismos endpoints en el mismo orden que las pantallas, incluidos los casos negativos —400, 401, 403, 404 y 409— y el contraste de permisos entre un administrador y una empresa.

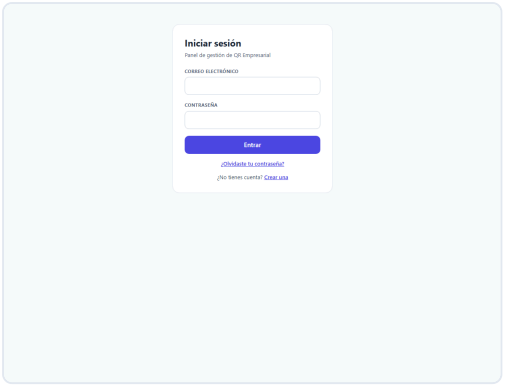
```
Guia React — Frontend N.1   3 peticiones   13 pruebas
Guia React — Frontend N.2  15 peticiones   31 pruebas
```

Crean su propia cuenta de prueba y la eliminan al terminar, de modo que dejan la base como la encontraron.

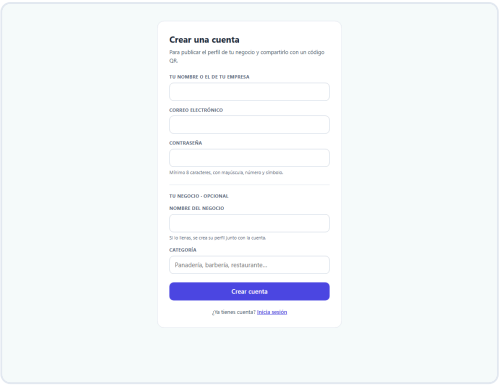
4. Capturas de la interfaz

Las dieciséis capturas siguientes las genera `frontend-react/evidencias/generar-capturas.js`, que abre un navegador, entra con las cuentas de demostración y recorre la aplicación **contra el servidor real**. Lo que se ve sale de MySQL: una pantalla dibujada con datos inventados no demuestra nada.

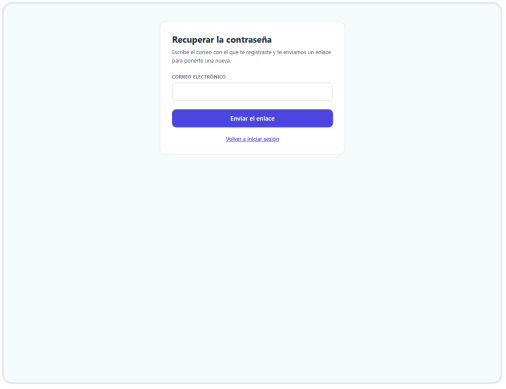
4.1 Acceso y cuenta



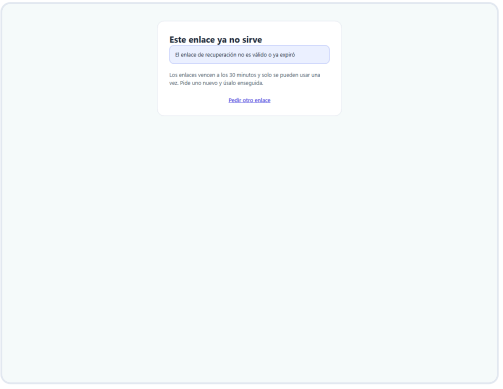
01 · Iniciar sesión. Guarda el token en `sessionStorage`: la sesión muere al cerrar la pestaña.



02 · Registro. El negocio es opcional; si se llena, la cuenta y su perfil se crean en una misma transacción.

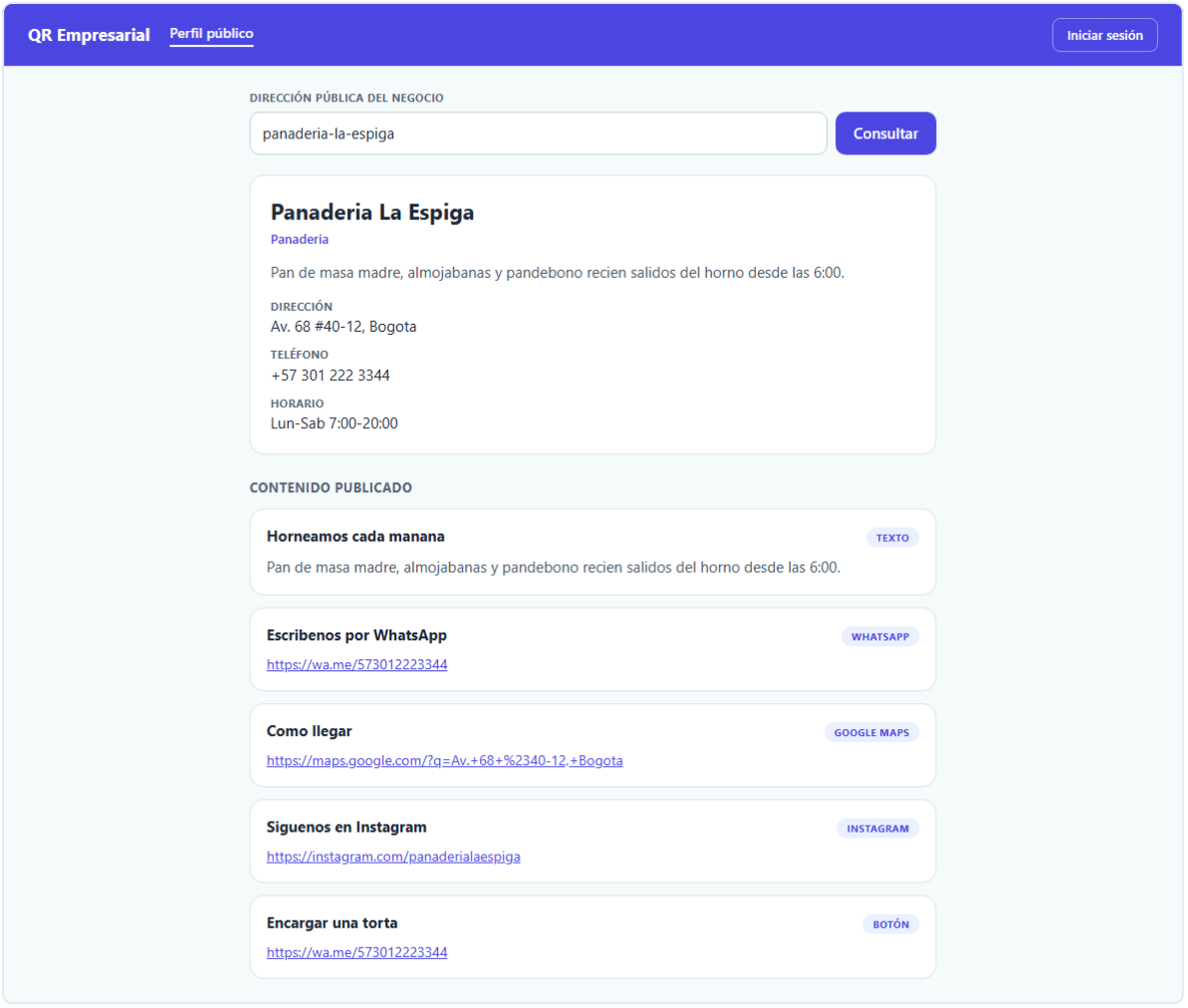


03 · Recuperar la contraseña. Responde lo mismo exista o no el correo, para no revelar qué direcciones están registradas.

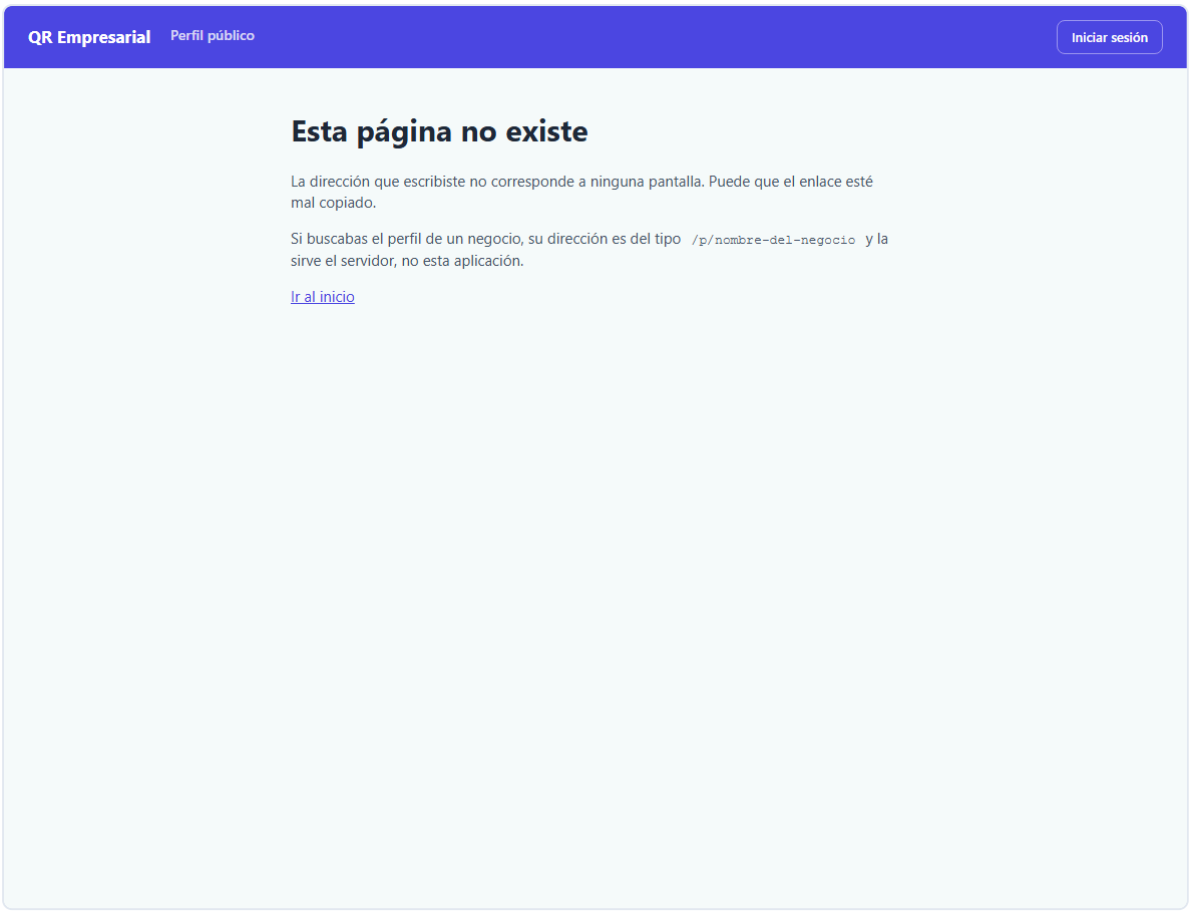


04 · Enlace caducado. Se comprueba antes de enseñar el formulario: pedir una contraseña nueva para luego decir que el enlace no sirve es la peor manera de contarlo.

4.2 Vista pública y errores

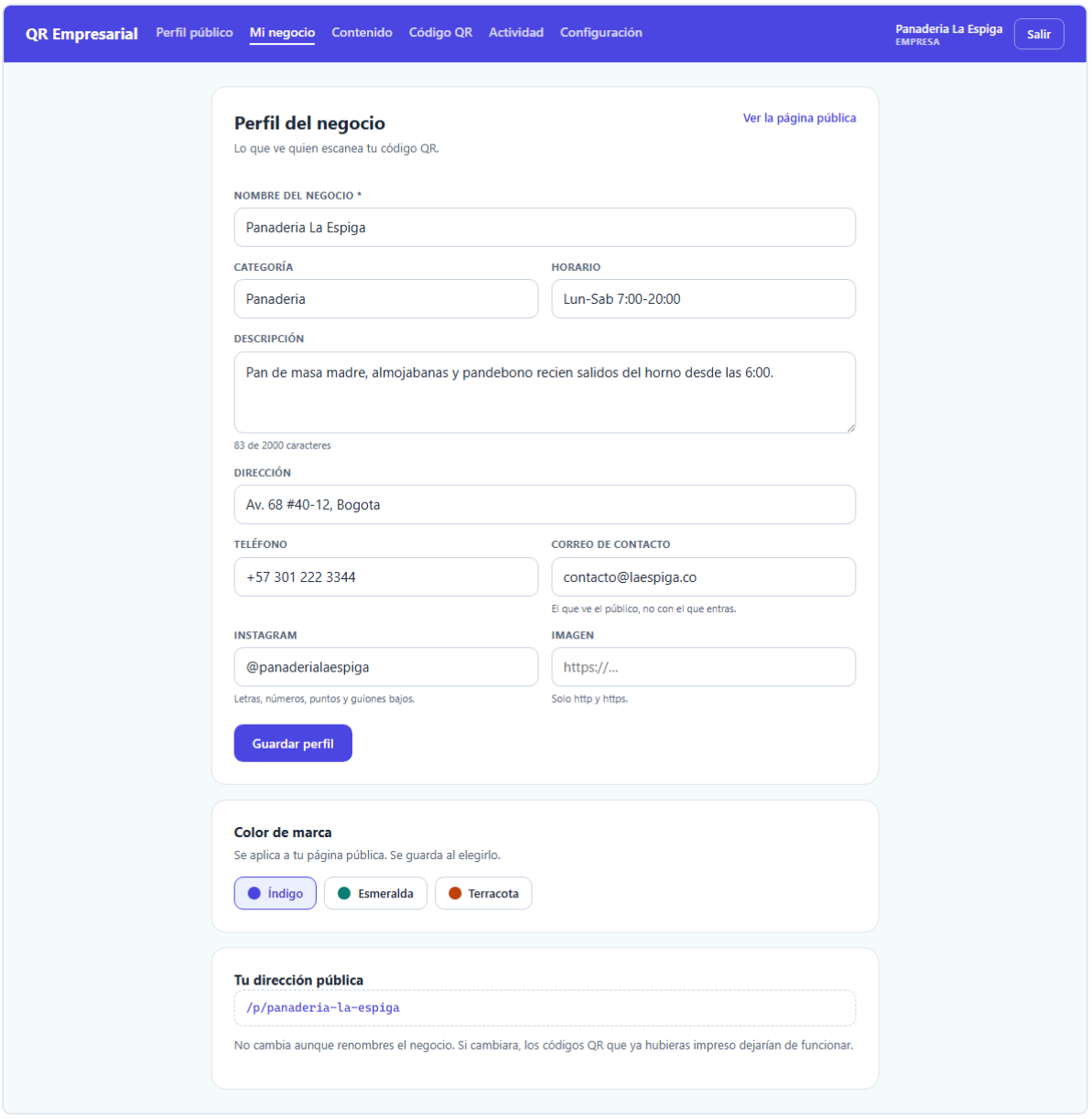


05 · Perfil público. Lo que ve quien escanea el código. No pide sesión. Solo aparecen los bloques activos: el dueño ve seis, el público cinco.

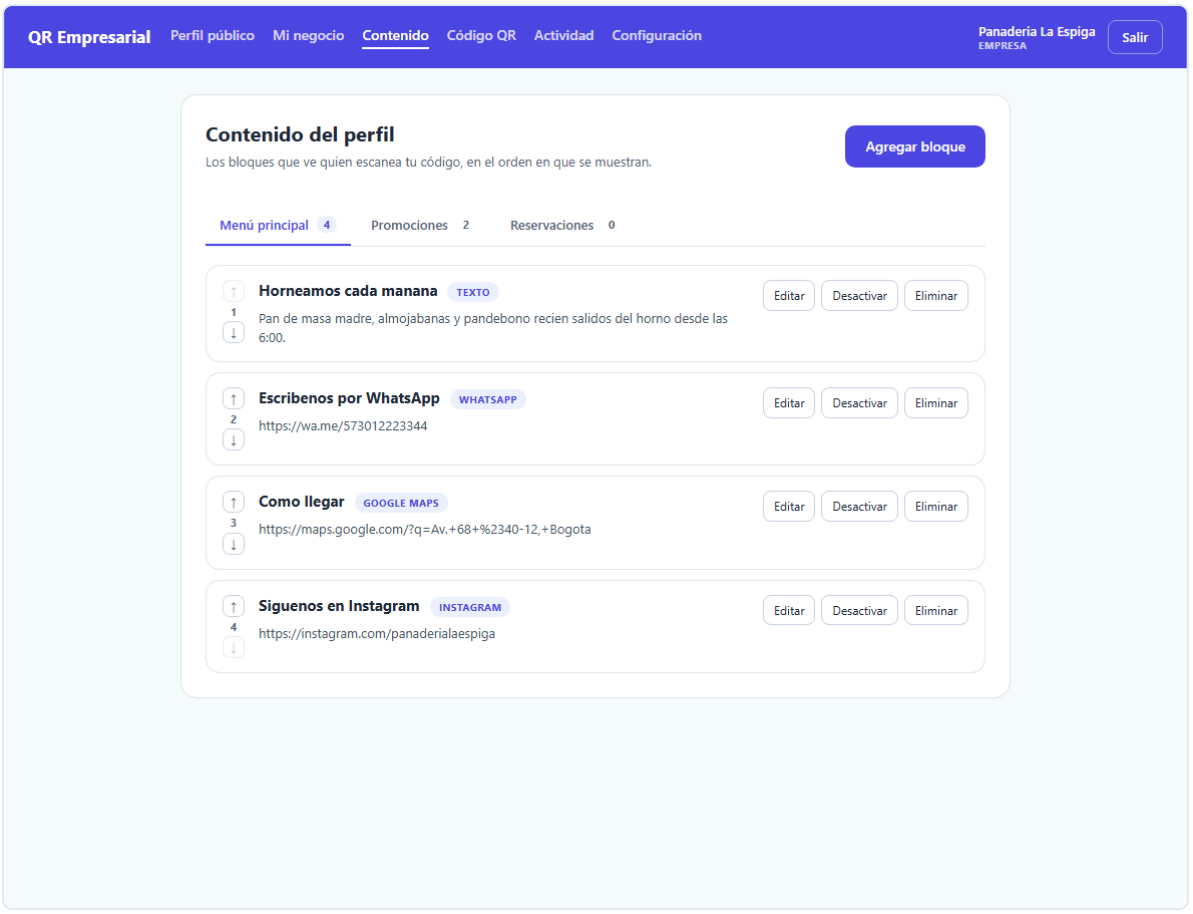


06 · Dirección inexistente. Se dice en lugar de redirigir a la portada en silencio, que haría creer al usuario que llegó donde quería.

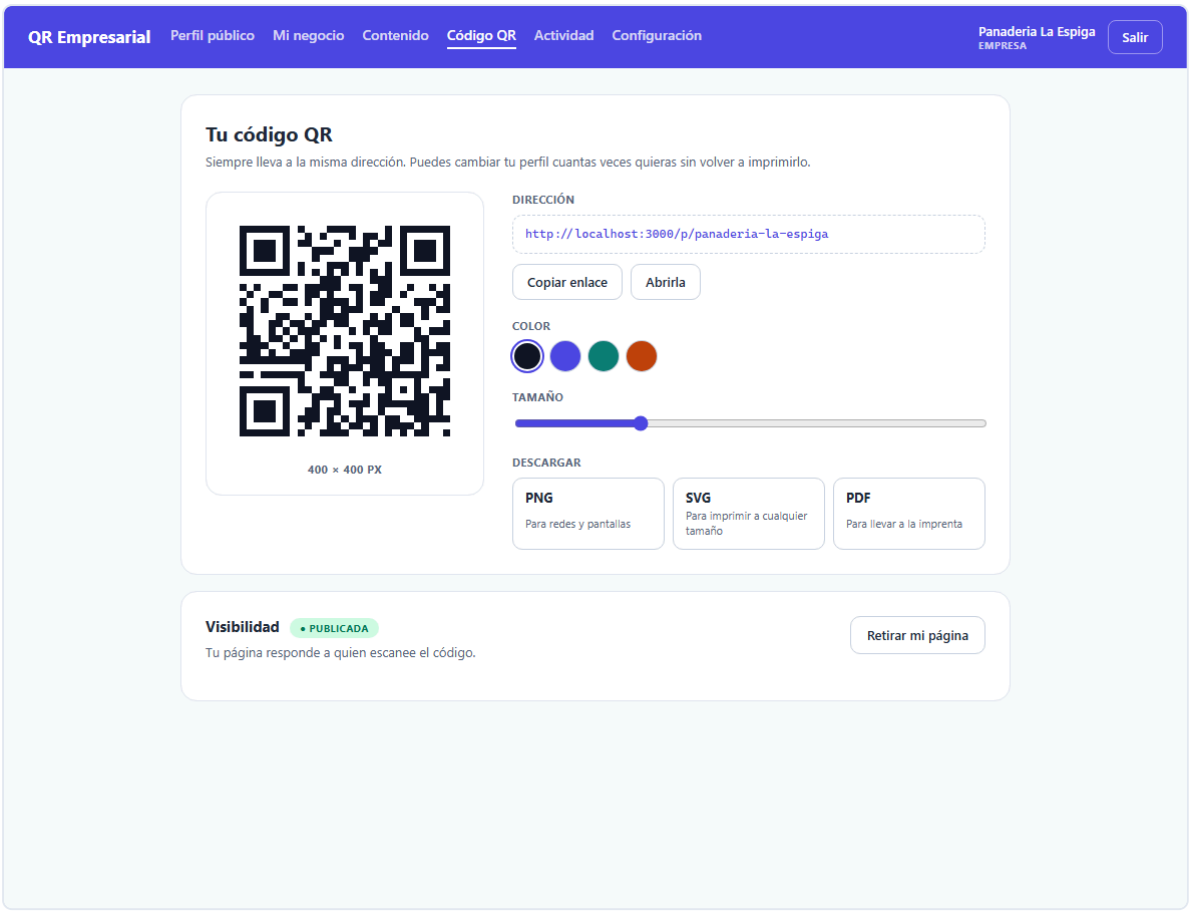
4.3 Panel del negocio



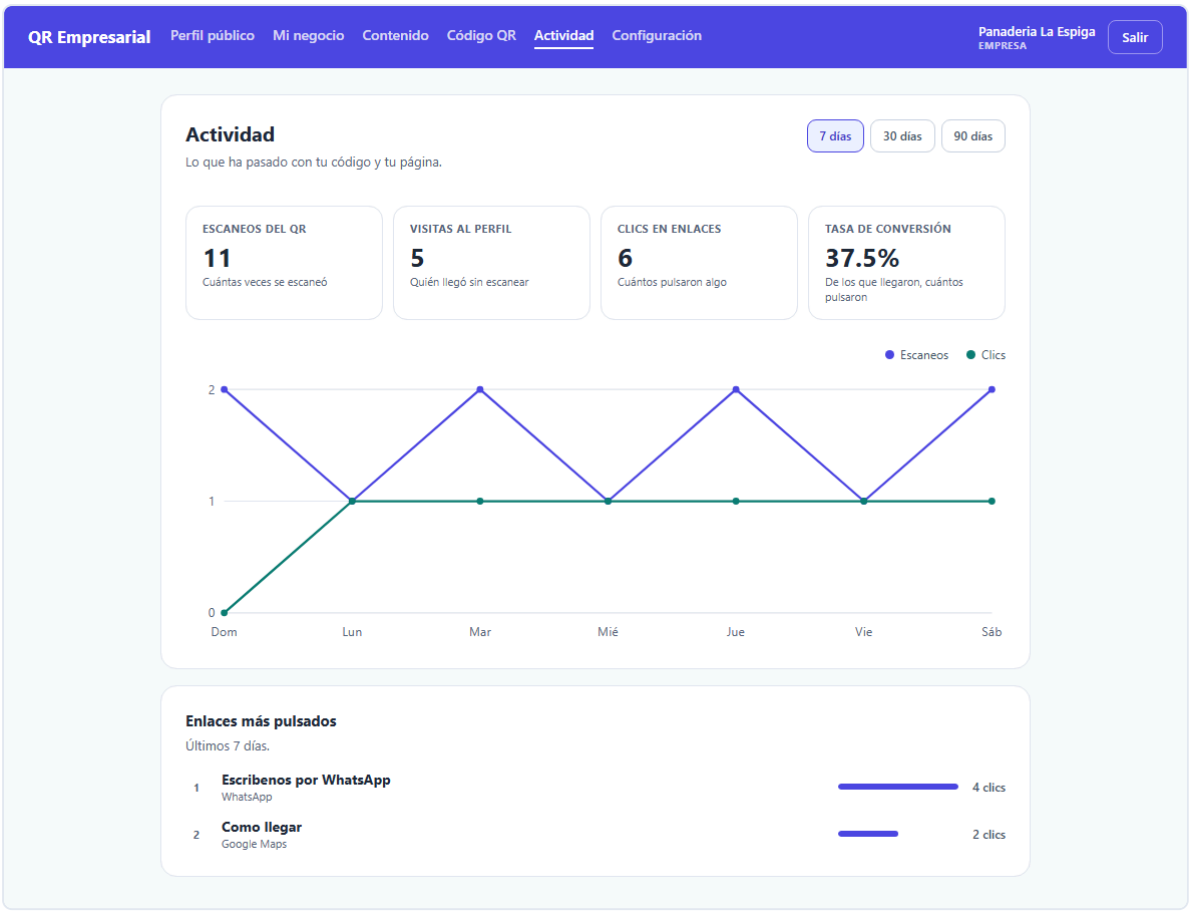
07 · Mi negocio. Los nueve campos del perfil, el color de marca y la dirección pública, que se muestra pero no se edita: si el `slug` cambiara, los códigos ya impresos dejarían de funcionar.



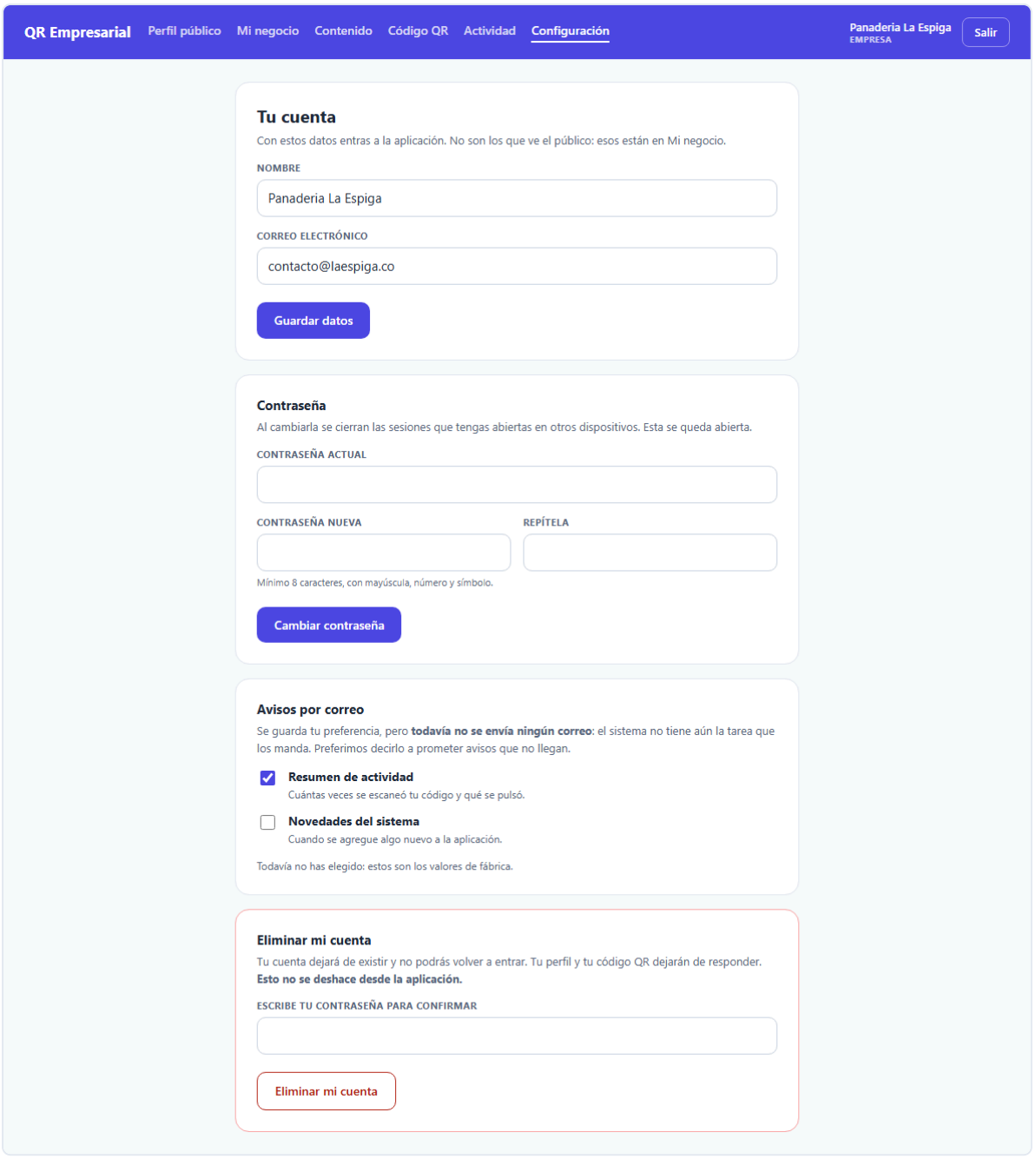
08 · Contenido. Tres secciones con su cuenta de bloques. Se reordena arrastrando y con las flechas: el arrastre de HTML no funciona con el dedo, y sin las flechas un móvil se quedaría sin forma de reordenar.



09 · Código QR. La vista previa la genera el servidor, el mismo endpoint que la descarga: lo que se ve es exactamente el archivo que se va a bajar.

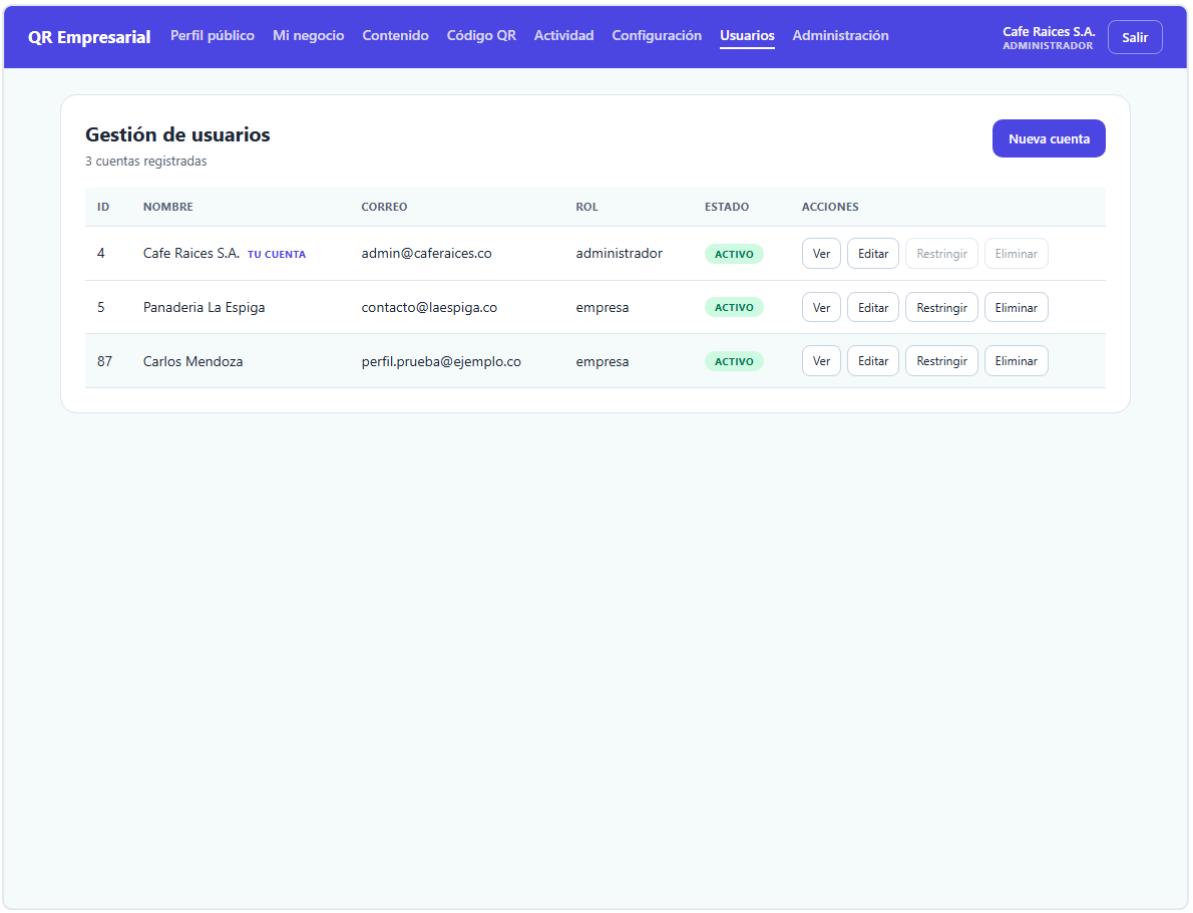


10 · Actividad. Cuatro medidas, la gráfica diaria en SVG y los enlaces más pulsados, en 7, 30 o 90 días —los tres rangos que acepta el servidor—.

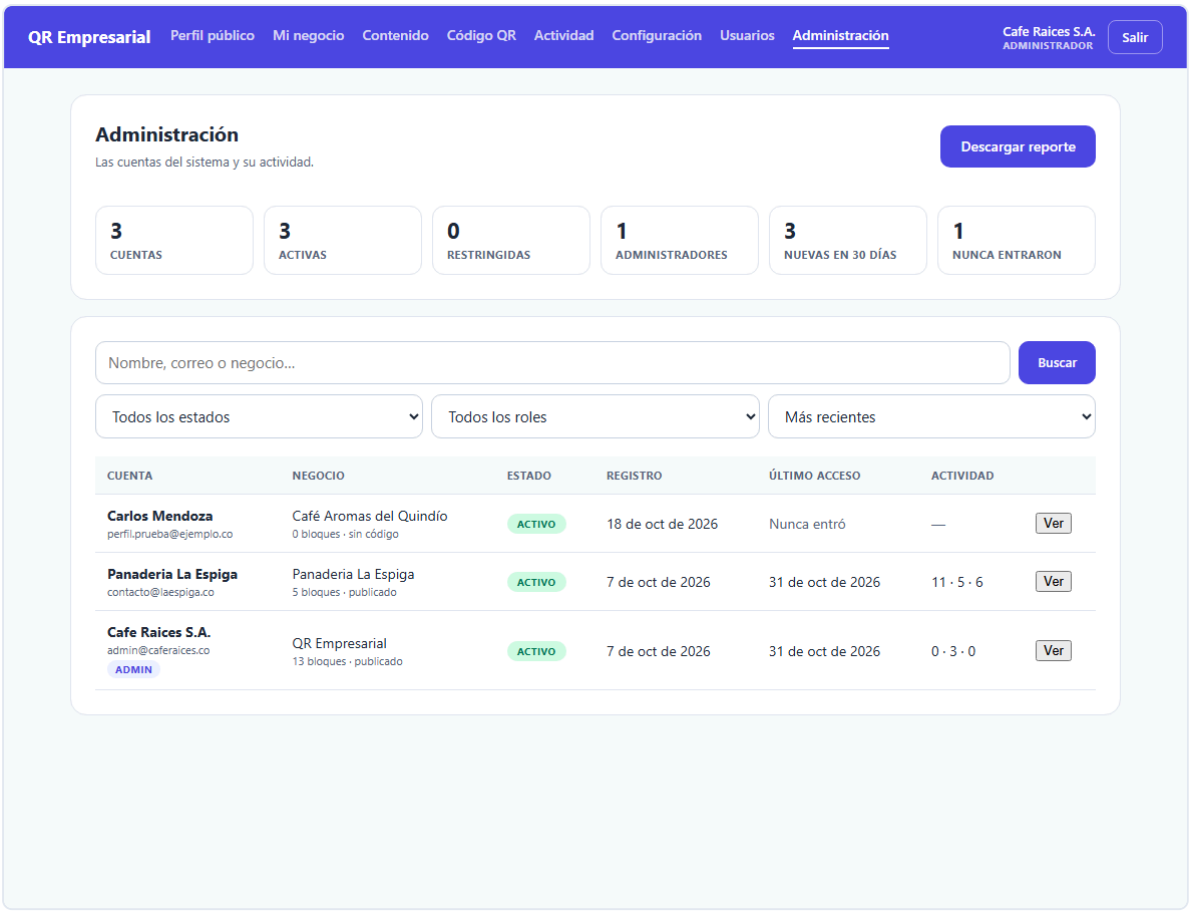


11 · Configuración. Datos, contraseña, avisos y baja. La contraseña solo se pide cuando cambia el correo. Los avisos se guardan pero no se envían, y la pantalla lo dice.

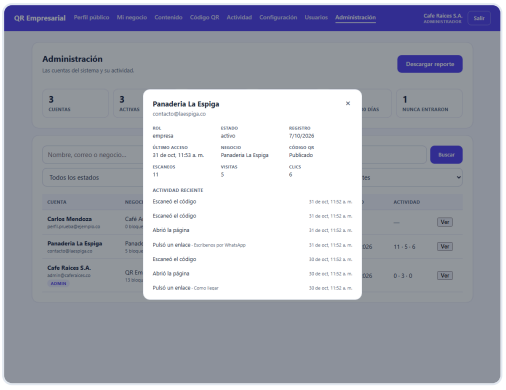
4.4 Administración



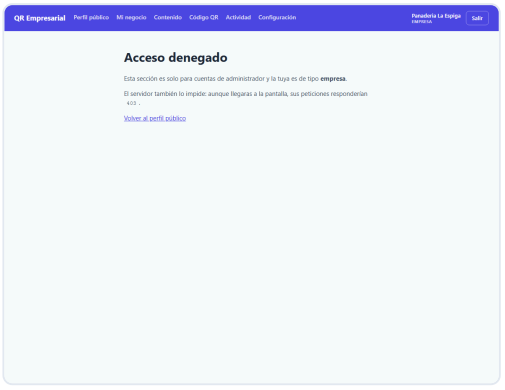
12 · Gestión de usuarios. Los siete endpoints del módulo. Restringir y eliminar están deshabilitados en la propia fila: el servidor lo impide y ofrecer un botón que solo puede fallar es una mala pantalla.



13 · Administración. Seis cifras, filtros con búsqueda, paginación y el reporte en CSV, que respeta los filtros y empieza por BOM para que Excel no rompa las tildes.

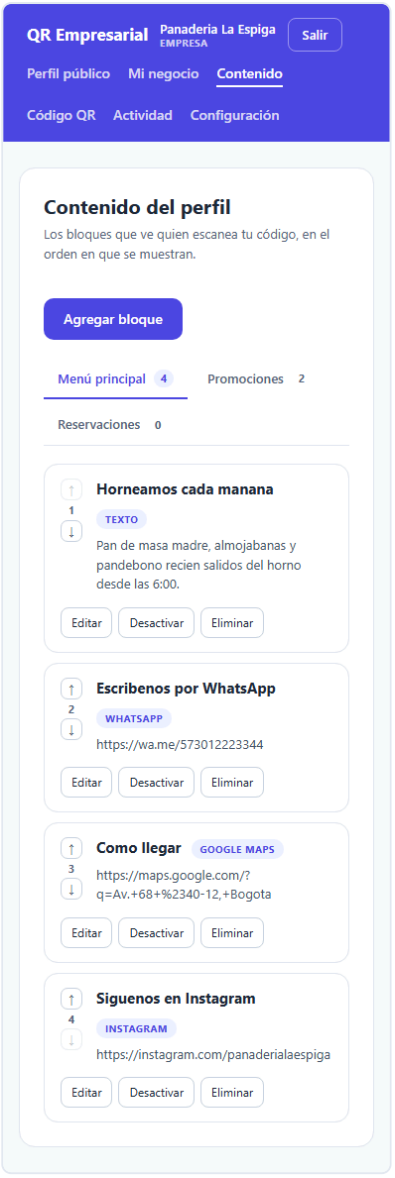


14 · Detalle de una cuenta. La actividad reciente, que la tabla no cabe.



15 · Acceso denegado. Una cuenta de empresa en una ruta de administrador.

4.5 Diseño adaptable



16 · Contenido a 390 px. Sin desplazamiento horizontal: la navegación baja a su propia línea y las acciones de cada bloque se reparten.

5. Repositorio Git

URL	https://github.com/JECAOR/qr-empresarial
Código del frontend	`frontend-react/`
README	`frontend-react/README.md` y el de la raíz
Evidencias	`frontend-react/evidencias/`

5.1 Historial de commits del frontend

Diecisiete commits sobre `frontend-react/`, del 21 al 23 de octubre de 2026, dentro de un repositorio con **120 commits y 47 etiquetas**. Cada módulo se desarrolló en su propia rama y se integró con `merge --no-ff`, de modo que la rama queda en el historial en lugar de aplanarse.

```
2026-10-23  feat(frontend-react): el modulo 8, la administracion          v3.0.0
2026-10-23  feat(frontend-react): el modulo 7, la cuenta                  v2.12.0
2026-10-23  feat(frontend-react): el modulo 6, la actividad del perfil    v2.11.0
2026-10-23  feat(frontend-react): el modulo 4, el codigo QR               v2.10.0
2026-10-23  feat(frontend-react): el modulo 3, el contenido del perfil    v2.9.0
2026-10-22  feat(frontend-react): el modulo 2, el perfil del negocio      v2.8.0
2026-10-22  feat(frontend-react): el modulo 1 completo                    v2.7.0
2026-10-22  docs(evidencias): las tres capturas que cierran la Guia 02    v2.6.0
2026-10-22  test(postman): carpeta de la Guia Frontend N.2                v2.5.1
2026-10-22  docs(evidencias): matriz de pruebas y capturas de la Guia 02
2026-10-22  feat(frontend-react): sesion JWT, rutas por rol y usuarios    v2.5.0
2026-10-21  docs(evidencias): la captura de Vite, Guia 01 completa        v2.4.0
2026-10-21  docs(evidencias): las cinco primeras capturas de la Guia 01
2026-10-21  docs(evidencias): Postman es una sola captura, no dos
2026-10-21  docs(frontend-react): carpeta de evidencias de la Guia 01
2026-10-21  docs(frontend-react): cierra los tres huecos de la Guia 01    v2.3.1
2026-10-21  feat(frontend-react): primera pantalla en React, Guia N.1     v2.3.0
```

Los mensajes siguen **Conventional Commits** y explican el porqué, no solo el qué. El historial completo está en GitHub.

5.2 Cómo ejecutarlo

```
mysql -u root -p --default-character-set=utf8mb4 qr_empresarial < database/seed.sql

cd backend        && npm install && npm run dev    # http://localhost:3000
cd frontend-react && npm install && npm run dev    # http://localhost:5173
```

Cuentas de demostración, las dos con la contraseña `Clave123*`: `admin@caferaices.co` (administrador) y `contacto@laespiga.co` (empresa).

5.3 Defectos encontrados y corregidos

Construir estas pantallas destapó cinco defectos, cuatro en el propio frontend y uno en el backend. Se documentan porque forman parte del proceso:

DEFECTO	CAUSA Y CORRECCIÓN
Un botón se publicaba como texto backend	El tipo se decidía comparando el nombre de la categoría con el literal `'Botón'`, y el cliente `mysql` de la consola de Windows reescribía los acentos al cargar las migraciones. Se corrigió con `SET NAMES utf8mb4` en los dieciséis archivos SQL y comparando por el icono, que es ASCII.
Nombres de columnas en la API backend	`GET /usuarios` devolvía las filas tal como salen de MySQL mientras el resto respondía en camelCase. Nadie lo veía porque el frontend original no llama a esa ruta.
El diálogo se cerraba solo	`close()` dispara el evento `close`, que borraba el estado; y `StrictMode` ejecuta el efecto dos veces. La limpieza sobraba.
El arrastre fallaba al ser rápido	El índice de origen iba en estado de React, que no se ha aplicado cuando llega el `drop` si los tres eventos caen en el mismo tic. Se pasó a `useRef`.
La barra se quedaba con el nombre viejo	Al cambiar el nombre, la barra seguía mostrando el anterior hasta recargar. `AuthContext` gana un `refrescar()`.

5.4 Lo que este frontend todavía no tiene

No hay pruebas automáticas propias. El frontend original del proyecto tiene 396 de Playwright; este se verificó pantalla por pantalla contra el servidor real, y las peticiones están cubiertas por las 44 pruebas de Postman, pero una suite de interfaz sigue pendiente.